

Hochschule für Technik und Wirtschaft Berlin

Probeklausur - Lösungsvorschlag

Lehrveranstaltungs-Nr.:	9191911
Lehrveranstaltung:	Datenbanken
Dozent/in:	Burak Boyaci
Sommer-/Wintersemester:	Wintersemester 2024
Studiengang:	Internationale Medieninformatik

Name, Vorname																			
---------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Matrikelnummer																			
----------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

1. Prüfen Sie, ob Ihre Klausur vollständig ist. Sie muss aus den durchnummerierten Blättern von 1 bis 10 bestehen. Nehmen Sie die Klausur bitte nicht auseinander. Falls Sie ein unvollständiges Exemplar erhalten haben, lassen Sie sich bitte eine einwandfreie Klausur aushändigen. Bitte nutzen Sie den vorgesehenen Platz für Ihre Lösungen. Sollte der gegebene Platz nicht ausreichen, verwenden Sie bitte jeweils die Rückseite der Aufgabenblätter.
2. Zum Bestehen der Klausur sind 50 % der Punktzahl erforderlich.
3. Die Bearbeitungszeit beträgt 90 Minuten.
4. Als Hilfsmittel ist ein handschriftlich beschriebenes 1-seitiges DIN-A4 Blatt erlaubt.
5. Das Mitführen eines Handys, Notebooks und anderen vergleichbaren elektronischen Gerätes in den Prüfungsraum während der Klausur stellt schon dann eine Täuschungshandlung dar, wenn der/die Student/in das Gerät ein- oder ausgeschaltet auf dem Arbeitsplatz liegen hat oder das Gerät ein- oder ausgeschaltet am Körper während der Prüfung bei sich führt. Das elektronische Gerät muss vor Beginn der Prüfung ausgeschaltet – nicht am Körper oder in der Kleidung – verstaut sein (sondern z. B. in einer verschlossenen Tasche) und darf während der gesamten Klausur nicht herausgenommen werden.
6. Schreiben Sie bitte gut leserlich und nicht mit Bleistift. Ihre Klausur wird ansonsten nicht gewertet.

Ich bestätige, dass ich prüfungsfähig bin und zu Beginn der Klausurarbeit die vollständigen Unterlagen erhalten zu habe.

Tag der Prüfung: _____

Unterschrift des Klausurteilnehmers/ der Klausurteilnehmerin

Aufgabe	1a	1b	1c	1d	2a	2b	2c	3a	3b	3c	3d	4a	4b	4c	4d	4e	5	Klausur	Übung	Gesamt
Punkte	2	2	2	4	10	5	5	4	2	2	2	4	4	4	4	4	10	70	30	100
Ergebnis	2	2	2	4	10	5	5	4	2	2	2	4	4	4	4	4	10	70	30	100

Punkte: _____ Note: _____

Unterschrift des Dozenten

Tag der Prüfung:

1. Mengenoperationen und allgemeine Verständnisfragen

Gegeben seien folgende beiden Relationen M und I:

M

<u>ID</u>	ModulName	Fachsemester	CreditPoints
1	Mathematik	3	4
2	Datenbanken	1	6
3	Data Science	2	5

I

<u>ID</u>	ModulName	Fachsemester	CreditPoints
4	Marketing	3	5
2	Datenbanken	1	6

a) Welches Ergebnis liefert folgende auf M und I angewendete Operation:

Select * from M
Union
Select * from I

<u>ID</u>	ModulName	Fachsemester	CreditPoints
1	Mathematik	3	4
2	Datenbanken	1	6
3	Data Science	2	5
4	Marketing	3	5

b) Welches Ergebnis liefert folgende auf M und I angewendete Operation:

```
Select * from M
Intersect
Select * from I
```

<u>ID</u>	ModulName	Fachsemester	CreditPoints
2	Datenbanken	1	6

c) Welches Ergebnis liefert folgende auf M und I angewendete Operation:

```
Select * from M
Except
Select * from I
```

<u>ID</u>	ModulName	Fachsemester	CreditPoints
1	Mathematik	3	4
3	Data Science	2	5

d) Wie bewerten Sie die folgenden 8 Aussagen. Entscheiden Sie, ob die Aussagen wahr (= w) oder falsch (= f) sind.

Aussage	wahr/falsch
Ein LIKE Operator funktioniert ausschließlich bei Strings	w
Ein PRIMARY KEY Constraint kombiniert NOT NULL und CHECK Constraints	f
Das kartesische Produkt einer 3x3 Relation mit einer 3x3 Relation ist wieder eine 3x3 Relation	f
Auf ein GROUP-BY folgt immer ein HAVING	f
Ein Vergleich mit einem NULL Wert ist immer unbekannt	w
Eine 1-N Kardinalität muss immer über eine Hilfstabelle aufgelöst werden	f
Eine Relation befindet sich in der 2NF, wenn sie sich in der 1NF befindet und alle Nichtschlüsselattribute vom gesamten Primärschlüssel unabhängig sind	f
In PL/pgSQL gibt eine Funktion ein Ergebnis zurück, eine Prozedur hingegen nicht	w

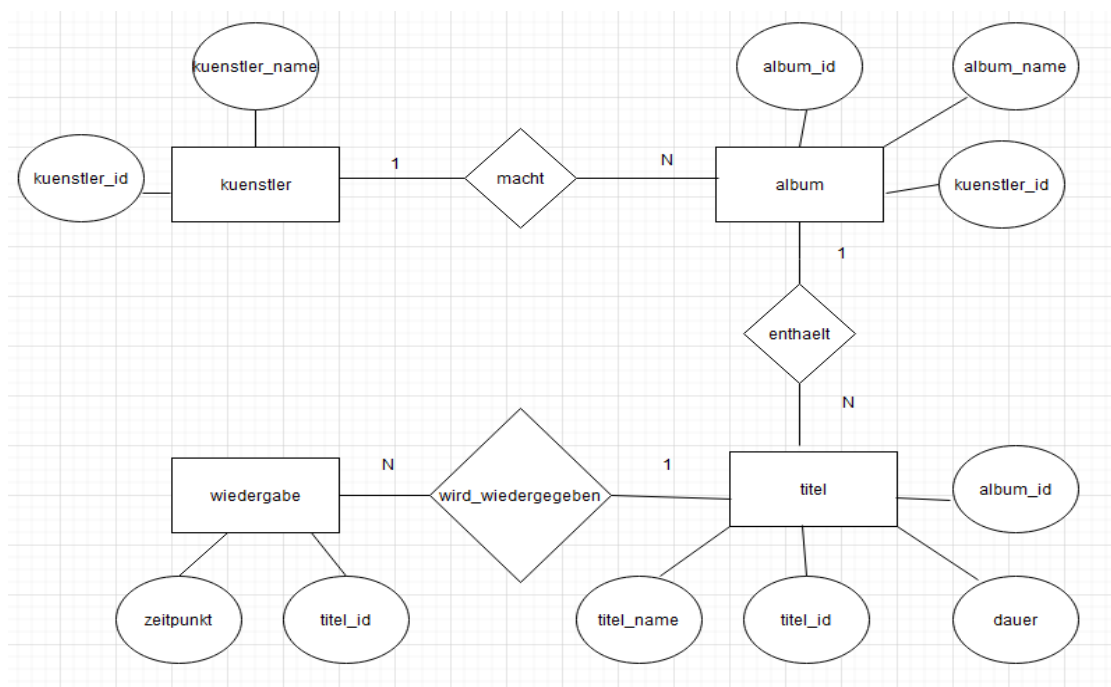
2. Datenmodellierung in Informationssystemen

Musik Apps wie Spotify oder Deezer speichern Informationen über Ihre Nutzer und erstellen basierend auf deren Vorlieben Profile Ihrer Nutzer. Im Auftrag eines Musik App Anbieters ist es Ihre Aufgabe eine kleine Datenbank zu erstellen, die Beziehungen zwischen Künstlern, Alben und Titeln speichert. Das Ergebnis ist eine persönliche Sammlung an Titeln.

a) Entity-Relationship-Diagramm

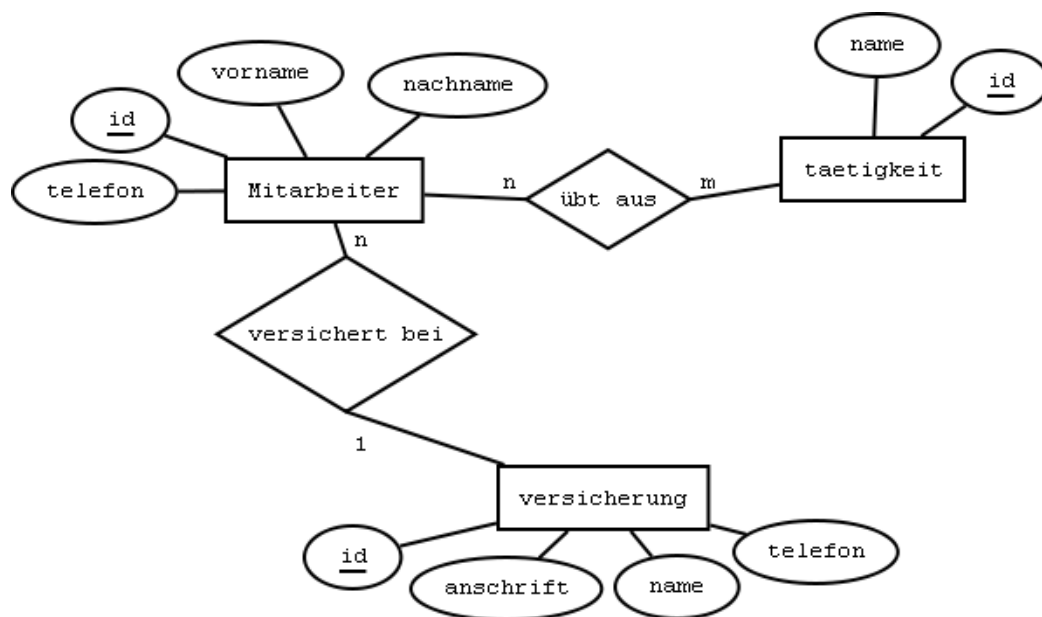
Skizzieren Sie bitte ein ER-Diagramm, das nachfolgende Sachverhalte wiedergeben kann. Vermerken Sie bei den Beziehungen auch die jeweils zutreffenden Kardinalitäten (1, M, N). Unterstreichen Sie in Ihrem Diagramm bitte Primärschlüssel mit einer durchgehenden Linie. Bitte verwenden Sie die in der Vorlesung verwendete CHEN-Notation.

1. Die Sammlung besteht aus Alben.
2. Ein Album wird von genau einem Künstler gemacht.
3. Ein Künstler kann ein oder mehrere Alben machen.
4. Ein Album enthält mindestens einen Titel oder auch mehrere Titel.
5. Künstler, Alben und Titel haben jeweils einen Namen.
6. Jeder Titel ist auf genau einem Album.
7. Jeder Titel hat eine Zeitdauer, gemessen in Sekunden.
8. Wenn ein Titel abgespielt wird, sollen Datum und Uhrzeit des Wiedergabebeginns aufgezeichnet werden. Dies wird verwendet, um zu melden, wann ein Titel zuletzt abgespielt wurde und wie oft Musik von einem Künstler, von einem Album oder einem Titel abgespielt wurde.



b) Relationales Datenbank-Modell

Überführen Sie bitte folgendes Entity-Relationship-Diagramm in eine relationale Datenbank. Sie sollten für jede Entität eine eigene Relation aufstellen. Bei bestimmten Kardinalitäten können zusätzliche Relationen erforderlich sein. Markieren Sie bitte in allen Relationen Primärschlüssel mit einer durchgezogenen Linie sowie Fremdschlüssel mit einer gestrichelten Linie.



Versicherung:

<u>id</u>	anschrift	name	telefon
-----------	-----------	------	---------

Mitarbeiter:

<u>id</u>	telefon	vorname	nachname
-----------	---------	---------	----------

Taetigkeit:

<u>id</u>	name
-----------	------

Mitarbeiter übt taetigkeit aus:

<u>ma_id</u>	<u>tae_id</u>
--------------	---------------

c) Normalisierung

Überführen Sie bitte folgende Tabelle in unnormalisierter Form (UNF) in die 3. Normalform (3NF).

KundenNr	Name	PLZ	Stadt	AuftrNr	AuftDatum	Beschreibung
1	DB GmbH	14167	Berlin	2	22.06.2023	Laminat verlegen
1	DB GmbH	14167	Berlin	1	31.05.2023	Wände streichen
2	Info AG	12459	Berlin	1	01.02.2023	Tür einbauen
2	Info AG	12459	Berlin	2	20.04.2023	Bad reinigen

Kunden:

KundenNr	Name	PLZ	Stadt
1	DB GmbH	14167	Berlin
2	Info AG	12459	Berlin

Aufträge:

AuftrNr	KundenNr	AuftDatum	Beschreibung
2	1	22.06.2023	Laminat verlegen
1	1	31.05.2023	Wände streichen
1	2	01.02.2023	Tür einbauen
2	2	20.04.2023	Bad reinigen

3. SQL Grundlagen

a) Erstellen Sie bitte die Tabelle Titel der Musik Datenbank aus Aufgabe 2a durch ein Create-Table-Statement, die durch folgende Spalten gekennzeichnet ist:

- Titel_ID
- Titel_Name
- Dauer

In dieser Tabelle sollen alle Einträge durch die Titel_ID eindeutig identifizierbar sein. Außerdem sollen für den Namen des Titels und die Dauer in dieser Tabelle keine leeren Werte eingefügt werden können. Datentypen sollen sinnvoll festgelegt werden.

```
CREATE TABLE Titel (
  Titel_ID SERIAL PRIMARY KEY,
  Titel_Name VARCHAR(255) NOT NULL,
  Dauer INT NOT NULL
);
```

- b) Sie stellen nach Anlage der Tabelle Titel fest, dass Sie noch eine Spalte Erscheinungsjahr benötigen, in der Sie festhalten, in welchem Jahr der Titel veröffentlicht wurde. Bitte erweitern Sie die Tabelle Titel um die Spalte Erscheinungsjahr und vergeben Sie einen geeigneten Datentyp.

```
ALTER TABLE Titel ADD COLUMN Erscheinungsjahr INT;
```

Im Nachfolgenden finden Sie einen Ausschnitt der Tabelle Titel:

	titel_id [PK] integer	titel_name character varying	dauer numeric	erscheinungsjahr character varying
1	1	Mensch	268	2002
2	2	Junge	187	2007
3	3	Cello	247	1973

- c) Im Nachhinein stellen Sie fest, dass der Titel mit der ID 3 eine Länge von 250 – und nicht von 247 Sekunden hat. Bitte ändern Sie den Wert in der Tabelle Titel durch einen geeigneten Befehl.

```
UPDATE Titel SET Dauer = 250 WHERE Titel_ID = 3;
```

- d) Bitte löschen Sie den Eintrag der ID 2 in der Tabelle Titel durch einen geeigneten Befehl.

```
DELETE FROM Titel WHERE Titel_ID = 2;
```

4. SQL Abfragen

Gegeben seien folgende beiden Tabellen:

Products:

	productcode [PK] text	productname text	productline text	productvendor text	quantityinstock integer	buyprice double precision	msrp double precision
1	S12_1666	1958 Setra Bus	Trucks and Buses	Welly Diecast Productions	1579	77.9	136.67
2	S12_3891	1969 Ford Falcon	Classic Cars	Second Gear Diecast	1049	83.05	173.02
3	S18_1097	1940 Ford Pickup Truck	Trucks and Buses	Studio M Art Models	2613	58.33	116.67
4	S18_1129	1993 Mazda RX-7	Classic Cars	Highway 66 Mini Classics	3975	83.51	141.54
5	S18_2238	1998 Chrysler Plymouth Prowler	Classic Cars	Gearbox Collectibles	4724	101.51	163.73
6	S18_2432	1926 Ford Fire Engine	Trucks and Buses	Carousel DieCast Legends	2018	24.92	60.77
7	S18_2625	1936 Harley Davidson El Knucklehead	Motorcycles	Welly Diecast Productions	4357	24.23	60.57
8	S18_2949	1913 Ford Model T Speedster	Vintage Cars	Carousel DieCast Legends	4189	60.78	101.31
9	S18_3029	1999 Yamaha Speed Boat	Ships	Min Lin Diecast	4259	51.61	86.02
10	S18_3140	1903 Ford Model A	Vintage Cars	Unimax Art Galleries	3913	68.3	136.59
11	S18_3278	1969 Dodge Super Bee	Classic Cars	Min Lin Diecast	1917	49.05	80.41
12	S18_3856	1941 Chevrolet Special Deluxe Cabriolet	Vintage Cars	Exoto Designs	2378	64.58	105.87
13	S18_4600	1940s Ford truck	Trucks and Buses	Motor City Art Classics	3128	84.76	121.08
14	S700_3505	The Titanic	Ships	Carousel DieCast Legends	1956	51.09	100.17
15	S18_4721	1957 Corvette Convertible	Classic Cars	Classic Metal Creations	1249	69.93	148.8

Orders:

	ordernumber [PK] integer	productcode [PK] text	quantityordered integer	priceeach double precision	orderlinenumber text
1	10101	S18_2795	26	167.06	1
2	10101	S24_2022	46	44.35	2
3	10101	S24_1937	45	32.53	3
4	10101	S18_2325	25	108.06	4
5	10102	S18_1367	41	43.13	1
6	10102	S18_1342	39	95.55	2
7	10103	S24_2300	36	107.34	1
8	10103	S18_1097	35	94.5	10
9	10103	S10_1949	26	214.3	11
10	10103	S18_2949	27	92.19	12
11	10103	S18_3136	25	86.92	13
12	10103	S18_2957	35	61.84	14
13	10103	S24_4258	25	88.62	15
14	10103	S18_3320	46	86.31	16
15	10103	S18_2432	22	58.34	2

- a) Schreiben Sie bitte einen SQL-Befehl, der alle *Produktcodes* mit dem entsprechenden *Produktnamen* und dem entsprechenden *Bestand* (= *quantityinstock*) für die Produkte ausgibt, die aus den Produktlinien *Classic Cars* sowie *Vintage Cars* stammen und die zusätzlich einen *Bestand* von weniger als 2000 Stück aufweisen. Das Ergebnis soll nach absteigendem *Bestand* sortiert werden.

```
SELECT productCode, productName, quantityInStock
FROM Products
WHERE productLine IN ('Classic Cars', 'Vintage Cars')
AND quantityInStock < 2000
ORDER BY quantityInStock DESC;
```

- b) Schreiben Sie bitte einen SQL-Befehl, mit dem Sie die *Gesamtmenge* aller *Bestände pro Produktlinie* bestimmen können und sortieren Sie das Ergebnis nach absteigender *Summe*. In der Ausgabe sollen die einzigartige *Produktlinie* und die *Summe* angezeigt werden.

```
SELECT productLine, SUM(quantityInStock) AS total_quantity
FROM Products
GROUP BY productLine
ORDER BY total_quantity DESC;
```

- c) Schreiben Sie bitte einen SQL-Befehl, mit dem Sie den maximalen *Kaufpreis* pro *Produktlinie* ermitteln können. In der Ausgabe sollen die einzigartige *Produktlinie* und der maximale *Kaufpreis* pro *Produktlinie* als *max* angezeigt werden. Das Ergebnis soll nach aufsteigendem *Maximum* sortiert angezeigt werden.

```
SELECT productLine, MAX(buyPrice) AS max
FROM Products
GROUP BY productLine
ORDER BY max;
```

- d) Schreiben Sie bitte einen SQL-Befehl, der alle *Bestellnummern* und die entsprechende *Anzahl* der bestellten Fahrzeuge (= *quantityordered*) für diejenigen *Bestellnummern* ausgibt, die das Produkt *The Titanic* enthalten und bei denen die *Anzahl* der bestellten Fahrzeuge mindestens 30 beträgt. In der Ausgabe sollen *Bestellnummern* und die *Anzahl* der Fahrzeuge angezeigt werden.

```
SELECT orderNumber, SUM(quantityOrdered) AS total_quantity
FROM Orders
WHERE productName = 'The Titanic'
AND quantityOrdered >= 30
GROUP BY orderNumber;
```

- e) Schreiben Sie bitte einen SQL-Befehl, der alle *Produktnamen* und den dazugehörigen *Kaufpreis* für diejenigen Produkte ausgibt, bei denen der *Kaufpreis* kleiner als der *durchschnittliche Kaufpreis* aller Produkte ist.

```
SELECT productName, buyPrice
FROM Products
WHERE buyPrice < (SELECT AVG(buyPrice) FROM Products);
```

5. PL/pgSQL – gespeicherte Prozedur

Schreiben Sie bitte eine gespeicherte Prozedur *Inflation* in PL/pgSQL, die folgendes tut: Bei Aufruf der Prozedur und Übergabe des Produktcodes als Parameter soll der Priceeach in der Tabelle *Orders* sich um 5% erhöhen. Außerdem soll auf der Konsole nach dem Aufruf der Prozedur folgende Information ausgegeben werden: 'Der Preis für das Produkt x hat sich um 5 Prozent erhöht.', wobei für x ein konkreter Produktcode stehen soll. Rufen Sie Ihre Prozedur anschließend bitte über einen geeigneten Befehl auf und übergeben Sie einen Productcode als Parameter.

```
CREATE OR REPLACE PROCEDURE Inflation(p_productCode TEXT)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE Orders
    SET priceEach = priceEach * 1.05
    WHERE productCode = p_productCode;
    RAISE NOTICE 'Der Preis für das Produkt % hat sich um 5 Prozent erhöht.',
    p_productCode;
END;
$$;

CALL Inflation('SOME_PRODUCT_CODE');
```